



Szoftver projekt laboratórium

11. ÖSSZESÍTETT DOKUMENTUM

Csapat

49 - komenymag

Konzulens

Marcsingó Ádám

Csapattagok

Görgényi András Levente	GSQEM0	andrisgorgenyi@gmail.com
Dulcz Bence	Y9LPT9	dulcz.bence@gmail.com
Varga Gergely	IBXDS0	vargageri05@gmail.com
Nagy Patrik	C3IJYQ	nagy.patrik0511@gmail.com
Tóth Dávid Pál	H9JYWA	tothdavid476@gmail.com

2026. május 10.

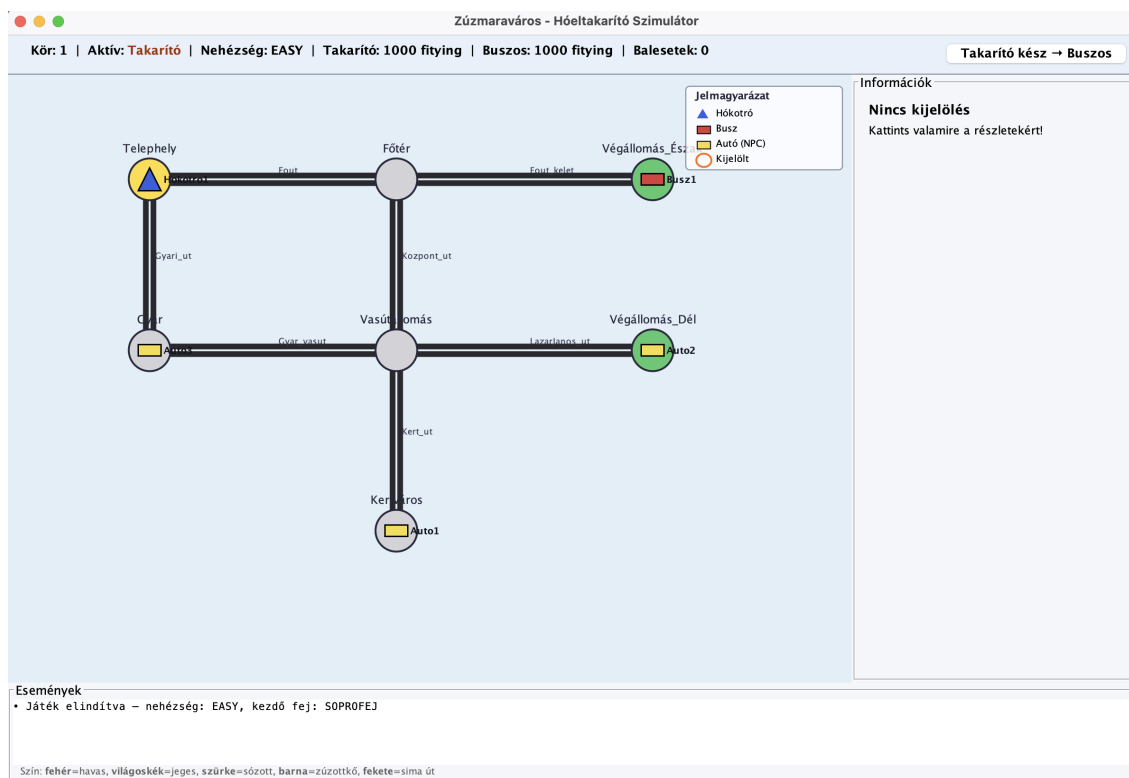
11. fejezet

Grafikus felület specifikációja

11.1. A grafikus interfész



11.1. ábra. Főmenü



11.2. ábra. Játékban

11.2. A grafikus rendszer architektúrája

11.2.1. A felület működési elve

A grafikus felület a meglévő üzleti logikát (motor, járművek, játékosok, takarítófejek, **terkep** csomagok) változtatás nélkül használja fel — a modell rétegben a GUI miatt csak additív bővítések történtek (új interfész, új getterek, új mezők), egyetlen meglévő metódus viselkedése sem módosult. Így a korábbi parancssoros prototípus ugyanúgy működik, mint korábban.

A három réteg felelőssége

A felület tiszta MVC elrendezést követ, három különálló csomagra bontva:

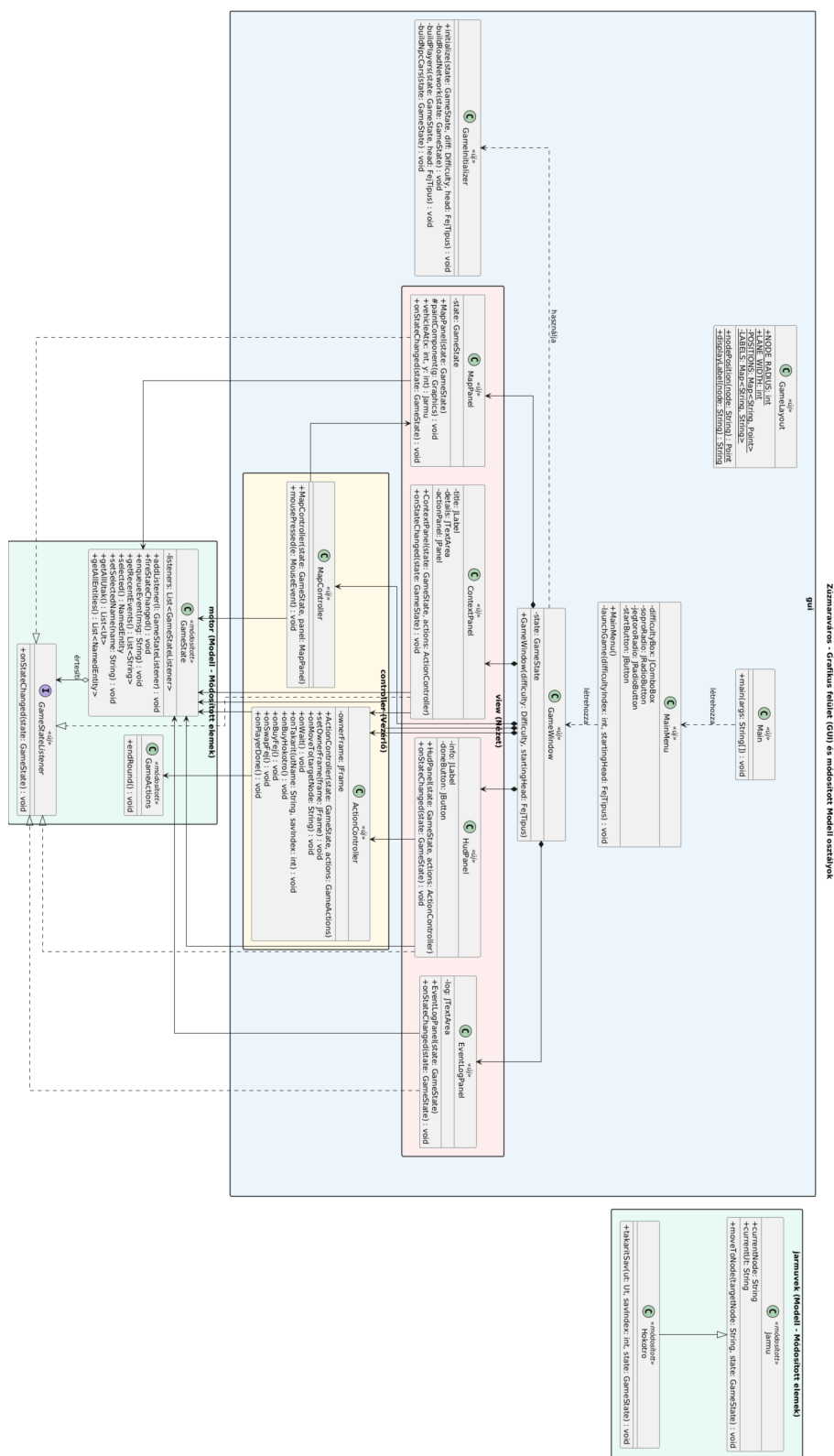
- **Modell** (motor, járművek, játékosok, takarítófejek, **terkep**) — az állapot és a játékszabályok.
- **Nézet** (gui.view) — a Swing-panelek (MapPanel, HudPanel, ContextPanel, EventLogPanel), amelyek a modell aktuális állapotát rajzolják. A nézet csak olvas a modellből, soha, sehol nem módosítja közvetlenül.
- **Vezérlő** (gui.controller) — a MapController és az ActionController. Ezek a felhasználói inputot fordítják le model-hívásokká (pl. `Hokotro.takaritSav`, `Jarmu.moveToNode`, `GameActions.endRound`).

A megjelenítés alapelve: kevert (push + pull)

Push oldal — eseményalapú értesítés. A motor csomagba felvettünk egy `GameStateListener` interfészt egyetlen `onStateChanged(GameState state)` metódussal. A `GameState` egy `addListener / fireStateChanged` páros segítségével

tartja számon a feliratkozott listenereket. Amikor a játékállapot ténylegesen változik (jármű kijelölése, lépés, takarítás, kör vége, vásárlás), egy magasabb szintű művelet végén — jellemzően az `ActionController`-ben vagy a `GameActions.endRound`-ban — a vezérlő meghívja a `state.fireStateChanged()`-et. A `GameWindow` minden view-panelt feliratkoztat listenerként, tehát az értesítés minden megjelenítendő komponensnek kézbesítődik.

Pull oldal — közvetlen lekérdezés rajzoláskor. A view-panelek `onStateChanged` implementációja nem ad át adatot magával az eseménnyel; egyszerűen kiváltja a saját frissítését (`repaint()` a `MapPanel`-en, vagy a label-ek és gombok újraépítése a `ContextPanel`-ben). A tényleges állapotot a panel a saját rajzolási vagy frissítési ciklusában közvetlenül olvassa le a modellből: a `MapPanel.paintComponent` például a `state.getAllUtak()`, `state.getAllEntities()`, `Sav.ho`, `Sav.ice` értékeit kérdezi le, a `ContextPanel` pedig a kiválasztott entitást a `state.selected()`-en keresztül. Így a megjelenítés mindig az aktuális, friss állapotot tükrözi anélkül, hogy a model-eseménynek bármilyen tartalmat kéne hordoznia.



11.3. ábra. Osztálydiagram

11.3. A grafikus objektumok felsorolása

11.3.1. Új osztályok

11.3.1.1. GameStateListener

■ Felelősség

Push-event interfész a model → view kommunikáció megvalósításához. Lehetővé teszi, hogy a modell értesítse a feliratkozott nézeteket (view-kat) az állapotváltozásról, így azok frissíthetik a megjelenített adatokat.

■ Interfészek

-

■ Ősosztály

-

■ Attribútumok

-

■ Metódusok

◇ *+onStateChanged()* : *void* - A modell hívja meg, amikor az állapota megváltozik, jelezve a nézetnek a grafikus frissítés szükségességét.

11.3.1.2. Main

■ Felelősség

A GUI alapú alkalmazás belépési pontja. Feladata a rendszerszintű megjelenés (Look & Feel) beállítása és a főmenü elindítása az Event Dispatch Thread-en (EDT).

■ Interfészek

-

■ Ősosztály

-

■ Attribútumok

-

■ Metódusok

◇ *+main(args : String[])* : *void* - A program belépési metódusa.

11.3.1.3. MainMenu

■ Felelősség

A játék indítóképernyője. Lehetővé teszi a nehézségi szint és a kezdő hókotrófej kiválasztását, majd elindítja a játéklablakot.

■ Interfészek

-

■ Ősosztály

JFrame

■ Attribútumok

- ◇ *-difficultyCombo* : *JComboBox* - Nehézség választó (Könnyű, Közepes, Nehéz).
- ◇ *-headRadioGroup* : *ButtonGroup* - Rádiógomb csoport a kezdő fej kiválasztásához (Söprő/Jégtörő).
- ◇ *-startButton* : *JButton* - A játék indítását kiváltó gomb.

■ Metódusok

- ◇ *+MainMenu()* - Konstruktor, felépíti a menü grafikus felületét.
- ◇ *-onStartGame()* : *void* - Eseménykezelő, amely létrehozza a GameWindow-t a választott paraméterekkel, majd bezárja a főmenüt.

11.3.1.4. GameWindow

■ Felelősség

A játék fő ablaka, amely összefogja a különböző nézeteket és kontrollereket. Összeállítja a játék kezdeti állapotát és kezeli a panelek elrendezését.

■ Interfészek

-

■ Ősosztály

JFrame

■ Attribútumok

- ◇ *-state* : *GameState* - A játék aktuális modell-állapota.
- ◇ *-mapPanel* : *MapPanel* - A térkép megjelenítő panel.
- ◇ *-hudPanel* : *HudPanel* - A felső információs sáv.
- ◇ *-contextPanel* : *ContextPanel* - A jobb oldali interakciós panel.
- ◇ *-eventLogPanel* : *EventLogPanel* - Az alsó eseménynapló panel.

■ Metódusok

- ◇ *+GameWindow(diff : Difficulty, head : HeadType)* - Létrehozza az ablakot, inicializálja a modellt és a nézeteket, majd feliratkoztatja a paneleket a modell eseményeire.

11.3.1.5. GameInitializer

■ Felelősség

Statikus segédosztály, amely a specifikáció szerinti várostérképet és a kezdő objektumokat (csomópontok, utak, járművek, NPC-k) hozza létre a memóriában.

■ Interfészek

-

■ Össosztály

-

■ Attribútumok

-

■ Metódusok

◇ *+createCity(state : GameState) : void* - Feltölti a kapott állapotot a hard-coded adatokkal: Telephely, Főtér, Vasútállomás, stb., valamint a játékosok és NPC-k kezdőhelyezeteivel.

11.3.1.6. GameLayout

■ Felelősség

A térkép statikus elhelyezkedéséért felelős segédosztály. Tárolja a koordinátákat és a megjelenítéshez szükséges konstansokat.

■ Interfészek

-

■ Össosztály

-

■ Attribútumok

◇ *+NODE_RADIUS : int* - Csomópontok rajzolási sugara.
 ◇ *+LANE_WIDTH : int* - Sávok szélessége pixelben.
 ◇ *-nodePositions : Map<String, Point>* - Csomópont azonosítóhoz rendelt koordináták.
 ◇ *-displayNames : Map<String, String>* - Belső azonosítók magyar megfelelői.

■ Metódusok

◇ *+getPosition(nodeId : String) : Point* - Visszaadja egy adott csomópont koordinátáját.
 ◇ *+getDisplayName(id : String) : String* - Visszaadja az azonosító magyar nevét.

11.3.1.7. MapPanel

■ Felelősség

A város térképének és a rajta lévő egységeknek a vizuális megjelenítése. Kirajzolja a csomópontokat, az utak állapotát és a járművek aktuális pozícióját.

■ Interfészek

GameStateListener

■ Össztály

JPanel

■ Attribútumok

◇ *-state : GameState* - Referencia a modellre az adatok lekéréséhez.

■ Metódusok

◇ *#paintComponent(g : Graphics) : void* - Kirajzolja a térképet: sárga/zöld/szürke csomópontok, utak a színkódolt állapotuk szerint, és a járművek ikonjai.

◇ *+vehicleAt(p : Point) : String* - Visszaadja a megadott koordinátán található jármű nevét, ha nincs ott semmi, null-t.

◇ *+onStateChanged() : void* - A modell jelzésére újrarajzolja a panelt (repaint).

11.3.1.8. HudPanel

■ Felelősség

A játék felső állapotsávja, amely az általános információkat (kör, aktív játékos, vagyon, balesetek) jeleníti meg.

■ Interfészek

GameStateListener

■ Össztály

JPanel

■ Attribútumok

◇ *-roundLabel : JLabel* - Kör sorszámát mutató felirat.

◇ *-wealthLabel : JLabel* - A játékosok vagyonát mutató felirat.

◇ *-doneButton : JButton* - A kört befejező "Kész vagyok" gomb.

■ Metódusok

◇ *+onStateChanged() : void* - Frissíti a feliratokat és a gomb szövegét az aktív játékos neve alapján.

11.3.1.9. ContextPanel

■ Felelősség

A kiválasztott jármű részleteit és a rajta végezhető akciókat (mozgás, takarítás, telepelyi műveletek) megjelenítő panel.

■ Interfészek

GameStateListener

■ Ősosztály

JPanel

■ Attribútumok

- ◇ *-actionContainer* : *JPanel* - Dinamikusan változó gombok tárolója.
- ◇ *-infoArea* : *JTextArea* - Jármű adatainak (üzemanyag, töltet) megjelenítése.

■ Metódusok

- ◇ *+onStateChanged()* : *void* - A kijelölt jármű típusa és helyzete alapján újragenerálja az akciógombokat.

11.3.1.10. EventLogPanel

■ Felelősség

A játék eseményeinek szöveges naplózása és a jelmagyarázat megjelenítése.

■ Interfészek

GameStateListener

■ Ősosztály

JPanel

■ Attribútumok

- ◇ *-logArea* : *JTextArea* - Az eseményeket listázó szövegdoboz.

■ Metódusok

- ◇ *+onStateChanged()* : *void* - Lekéri a modell legfrissebb eseményeit és megjeleníti azokat.

11.3.1.11. MapController

■ Felelősség

A térképen történő egérekattintások kezelése, a járművek kijelölésének és deszelektálásának vezérlése.

■ Interfészek

MouseListener

■ Össosztály

MouseListener

■ Attribútumok

- ◇ *-mapPanel : MapPanel* - A megfigyelt térkép panel.
- ◇ *-state : GameState* - A modell, ahol a kijelölést állítja.

■ Metódusok

- ◇ *+mouseClicked(e : MouseEvent) : void* - Kattintáskor a vehicleAt segédmetódussal lekérdezi, van-e ott jármű, és frissíti a modellben a kijelölt objektumot.

11.3.1.12. ActionController

■ Felelősség

A felhasználói felületről érkező gombnyomásokat fordítja le konkrét modell-műveletekre, kezeli a körök közötti váltást és a játék végének logikáját.

■ Interfészek

-

■ Össosztály

-

■ Attribútumok

- ◇ *-state : GameState* - A játék modell-állapota.

■ Metódusok

- ◇ *+onMoveTo(roadId : String) : void* - A kijelölt jármű mozgatása a kiválasztott útra.
- ◇ *+onTakarit() : void* - A takarítási akció elindítása a felszerelt fejnek megfelelően.
- ◇ *+onBuyHokotro() : void* - Új hókotró vásárlása a telephelyen.
- ◇ *+onPlayerDone() : void* - Az aktív játékos körének lezárása, váltás a következő játékosra, vagy NPC körök futtatása.
- ◇ *-ensureControllable() : boolean* - Ellenőrzi, hogy a kijelölt jármű az aktív játékos-e.
- ◇ *-showGameOver(winner : String) : void* - Megjeleníti a játék vége dialógust a győztesrel.

11.3.2. Megváltozott osztályok

11.3.2.1. GameState

■ Felelősség

A játék teljes állapotát reprezentáló osztály, amely a módosítások során kibővült a nézetek értesítéséért felelős push-mechanizmussal (Listener minta), BFS alapú útvonalkereséssel az NPC-k számára, valamint a Pass'N'Play körökre bontott vezérléshez szükséges állapotokkal és a UI eseménynapló kezelésével.

■ Interfészek

-

■ Össztály

-

■ Attribútumok

- ◇ *+currentRound* - Körszámláló, értéke 1-től indul.
- ◇ *+activePlayerName* - A Pass'N'Play módban éppen aktív játékos neve.
- ◇ *+gameOverReason* - Magyar nyelvű szöveges üzenet a játék végének okáról.
- ◇ *+running* - Jelzi, hogy a játék fut-e még (korábban package-private, most publikus a HUD és ActionController miatt).
- ◇ *-recentEvents* - A legutóbbi UI eseményeket tároló adatstruktúra (maximum 12 esemény, FIFO).

■ Metódusok

- ◇ *+addListener(l : GameStateListener) : void* - Feliratkoztat egy új nézetet az állapotváltozásokra.
- ◇ *+removeListener(l : GameStateListener) : void* - Eltávolít egy feliratkozott nézetet a listából.
- ◇ *+fireStateChanged() : void* - Hibatűró módon (kivételkezeléssel) push-értesítést küld minden feliratkozott listenernek a változásokról.
- ◇ *+setSelectedName(name : String) : void* - Beállítja a kiválasztott entitást, és automatikusan meghívja a *fireStateChanged()* metódust.
- ◇ *+getAllUtak() : Collection* - Publikus getter, amely a GUI számára olvashatóvá teszi az utakat.
- ◇ *+getAllEntities() : Collection* - Publikus getter az összes entitás lekérdezéséhez.
- ◇ *+getDepotNode() : String* - Visszaadja a telephely csomópontjának azonosítóját.
- ◇ *+shortestPath(from : String, to : String) : List* - BFS alapú útvonalkereső, visszaadja a legrövidebb utat két csomópont között.
- ◇ *+nextStepToward(from : String, to : String) : String* - A BFS algoritmus alapján meghatározza a következő lépéshez szükséges csomópontot a cél felé.
- ◇ *+roadBetween(nodeA : String, nodeB : String) : Ut* - Visszaadja a megadott két csomópontot összekötő utat.
- ◇ *+switchActivePlayer() : void* - Átvált a következő játékosra a Pass'N'Play logikának megfelelően.
- ◇ *+resetActivePlayer() : void* - Visszaállítja a kezdő játékost aktívnak.
- ◇ *+canControl(j : Jarmu) : boolean* - Segédmetódus, eldönti, hogy az aktuálisan aktív játékos irányíthatja-e a kapott járművet.

- ◇ *+getRecentEvents()* : *Deque* - Publikusan elérhetővé teszi az eseménynapló tartalmát a UI számára.
- ◇ *+enqueueEvent(msg : String)* : *void* - Beleteszi az új eseményt a recentEvents tárolóba is (max. 12 elem, FIFO elv alapján).

11.3.2.2. GameActions

■ Felelősség

A játék logikai műveleteit összefogó osztály, amely kiegészült a körök lezárását és az NPC-k automatikus mozgását végző logikával.

■ Interfészek

-

■ Ősosztály

-

■ Attribútumok

-

■ Metódusok

- ◇ *+endRound()* : *void* - Lezárja a kört: végrehajtja az összes NPC autó npcStep() lépését, minden úton alkalmaz egy egységnyi havazást, növeli a körszámlálót (currentRound), frissíti az időt, kiértékeli a játékvége feltételeket, és végül frissíti a nézeteket a fireStateChanged() hívásával.

11.3.2.3. Auto

■ Felelősség

Az utakon közlekedő NPC autókat reprezentáló osztály, amely megkapta a legrövidebb úton (BFS) történő önálló navigációhoz szükséges célpontokat és döntési logikát.

■ Interfészek

-

■ Ősosztály

Jarmu

■ Attribútumok

- ◇ *+otthon* - Az NPC autó otthonát jelentő csomópont.
- ◇ *+munkahely* - Az NPC autó munkahelyét jelentő csomópont.
- ◇ *-aktualisCel* - Az autó éppen aktuális úticélja (otthon vagy munkahely).
- ◇ *-utolsoCsomopont* - A legutóbb elhagyott csomópont azonosítója a belső állapottartáshoz.

■ Metódusok

- ◇ *+setupRoute*(*otthon* : *String*, *munkahely* : *String*, *startNode* : *String*) : *void*
- Inicializálja az autó kezdő paramétereit és útvonalát a létrehozáskor.
- ◇ *+getAktualisCel*() : *String* - Visszaadja az autó jelenlegi célállomását.
- ◇ *+npcStep*(*state* : *GameState*) : *void* - Az NPC autó tesz egy lépést a célja felé a legrövidebb úton. Tartalmazza a hibakezelést: ha a kinézett sáv járhatatlan, az autó a csomóponton várakozik.
- ◇ *-closerNodeToward*(*node1* : *String*, *node2* : *String*, *target* : *String*) : *String*
- Privát segédmetódus, amely egy adott út két végpontja közül kiválasztja a célhoz közelebb esőt.

11.3.2.4. Busz

■ Felelősség

A busz specifikus viselkedését megvalósító osztály. A módosítások a rugalmasabb (RegEx / minta alapú) végállomás-felismerést vezették be a statikus string egyezés helyett.

■ Interfészek

-

■ Össztály

Jarmu

■ Attribútumok

-

■ Metódusok

- ◇ *-isVegallomasNode*(*nodeName* : *String*) : *boolean* - Statikus segédmetódus, amely ellenőrzi, hogy a megadott csomópont neve „Vegallomas” vagy „Vegallomas_” mintázatú-e.
- ◇ *+onCelUtElerve*(*target* : *String*, *state* : *GameState*) : *void* - Felüldefiniált eseménykezelő. Most már az *isVegallomasNode* helper alapján bármely „Vegallomas_” csomópontnál sikeresen regisztrálja a busz körét.

11.3.2.5. Hokotro

■ Felelősség

A játékos által irányított hókotrót reprezentáló osztály. A változtatások a fej (Söprő/Jégtörő) közvetlen lekérdezését és beállítását teszik lehetővé a grafikus felület és a kezdeti beállítások számára.

■ Interfészek

-

■ Össztály

Jarmu

■ Attribútumok

-

■ Metódusok

◇ *+getAktivFej()* : *FejTipus* - Visszaadja a hókotróra jelenleg felszerelt aktív fejet (a ContextPanel és ActionController használja).

◇ *+setAktivFej(fej : FejTipus)* : *void* - Beállítja a hókotró aktív fejét (a főmenü kezdő beállításához és az új hókotró vásárlásához szükséges).

11.3.2.6. Jatekos

■ Felelősség

A játékost és a hozzá tartozó erőforrásokat (vagyon, raktár) reprezentáló osztály. Új funkciója a raktárkészlet biztonságos expozíciója a nézet réteg felé.

■ Interfészek

-

■ Össztály

-

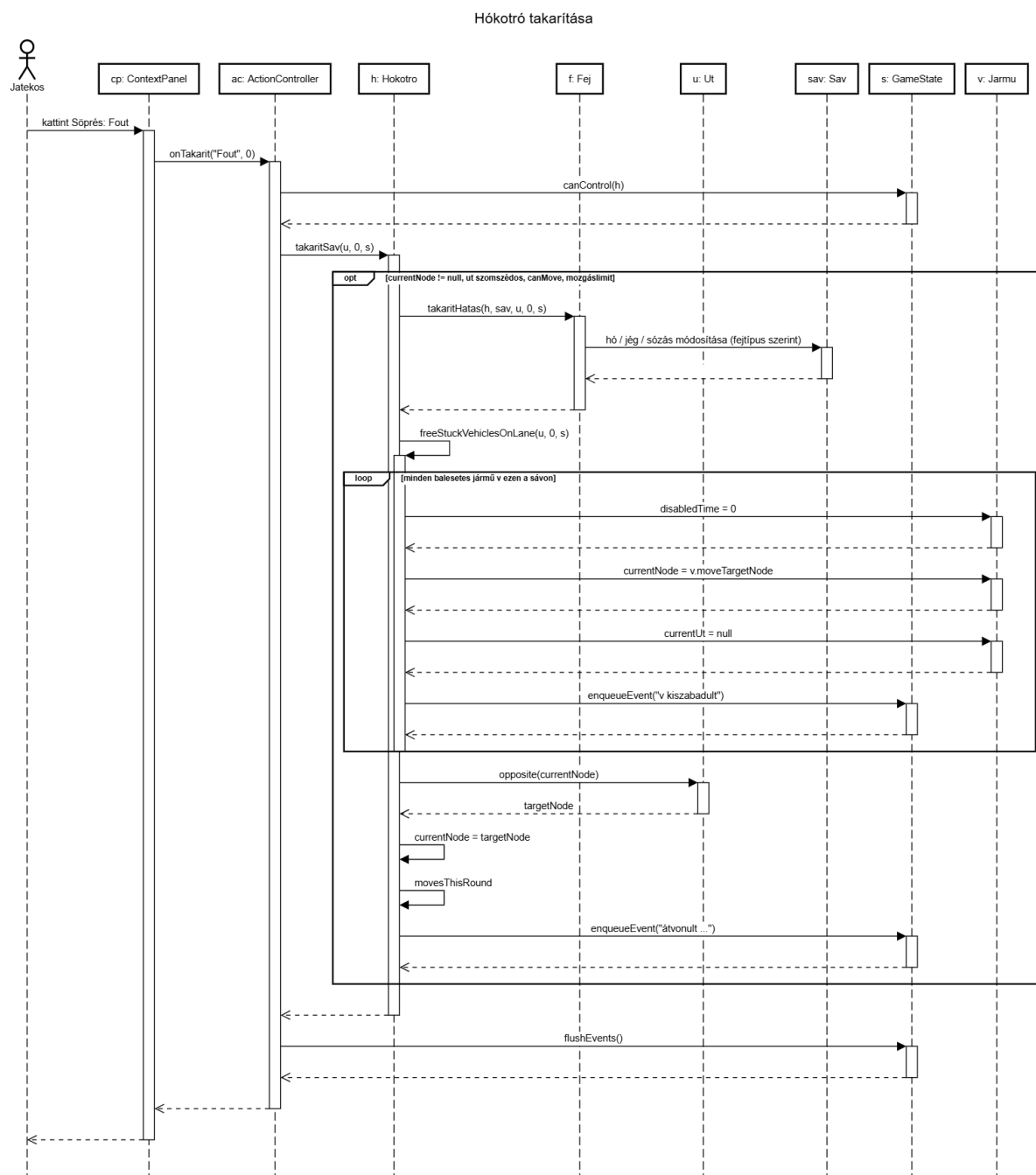
■ Attribútumok

-

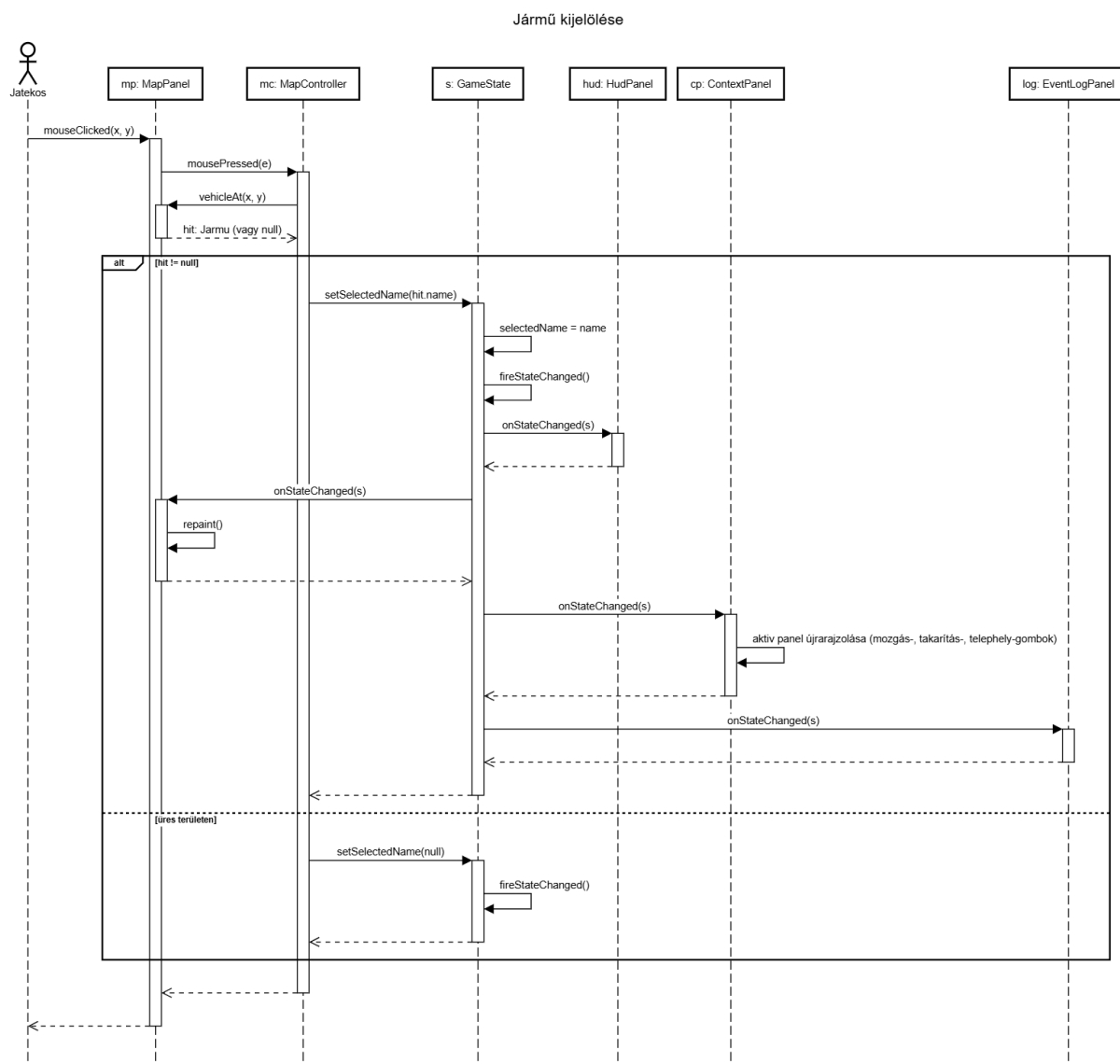
■ Metódusok

◇ *+getFejInventory()* : *List* - Módosíthatatlan (read-only) nézetet ad vissza a játékos birtokában lévő hókotró fejekekről, amelyet a GUI (pl. fejcsere dialógus) olvas.

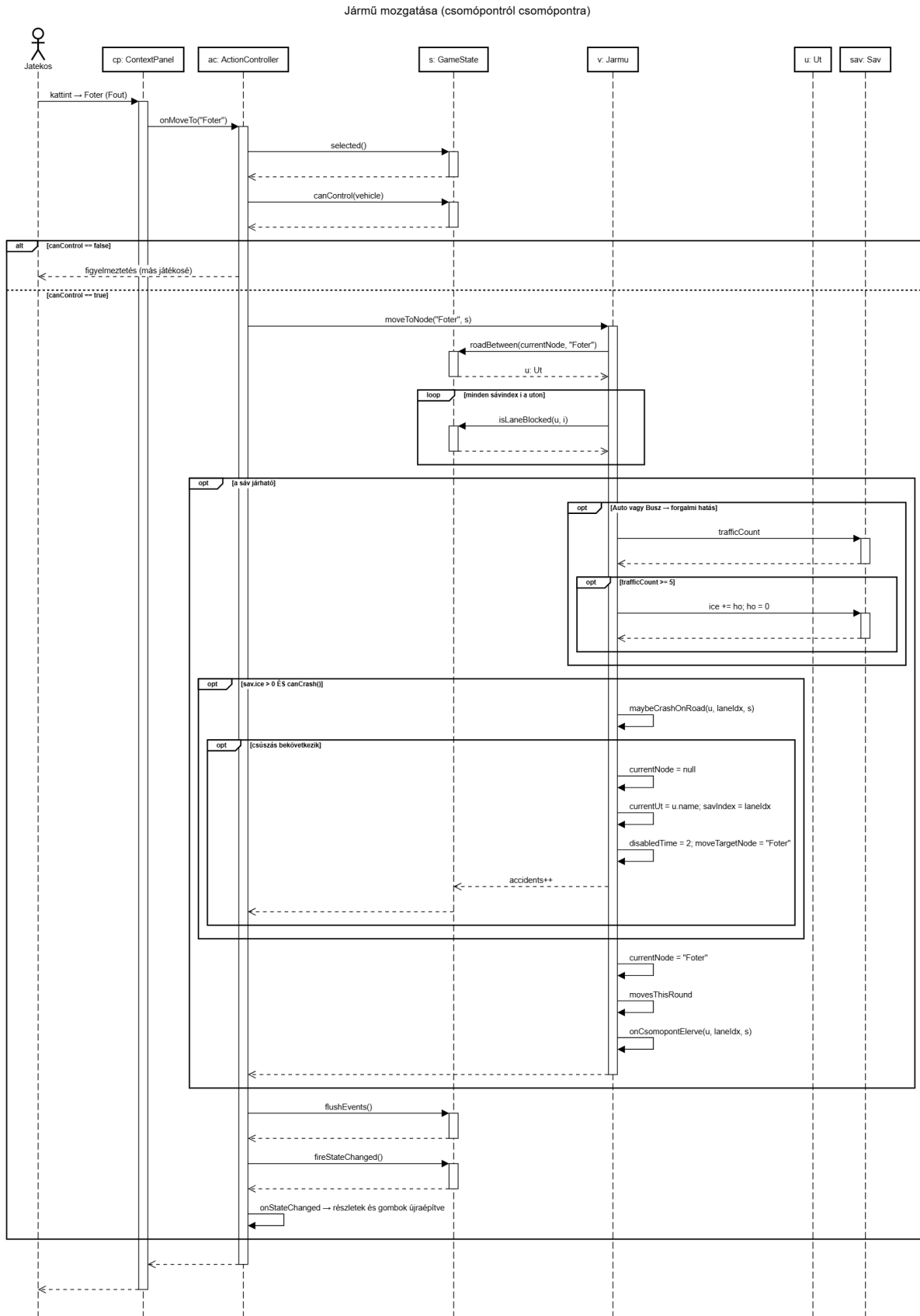
11.4. Kapcsolat az alkalmazói rendszerrel



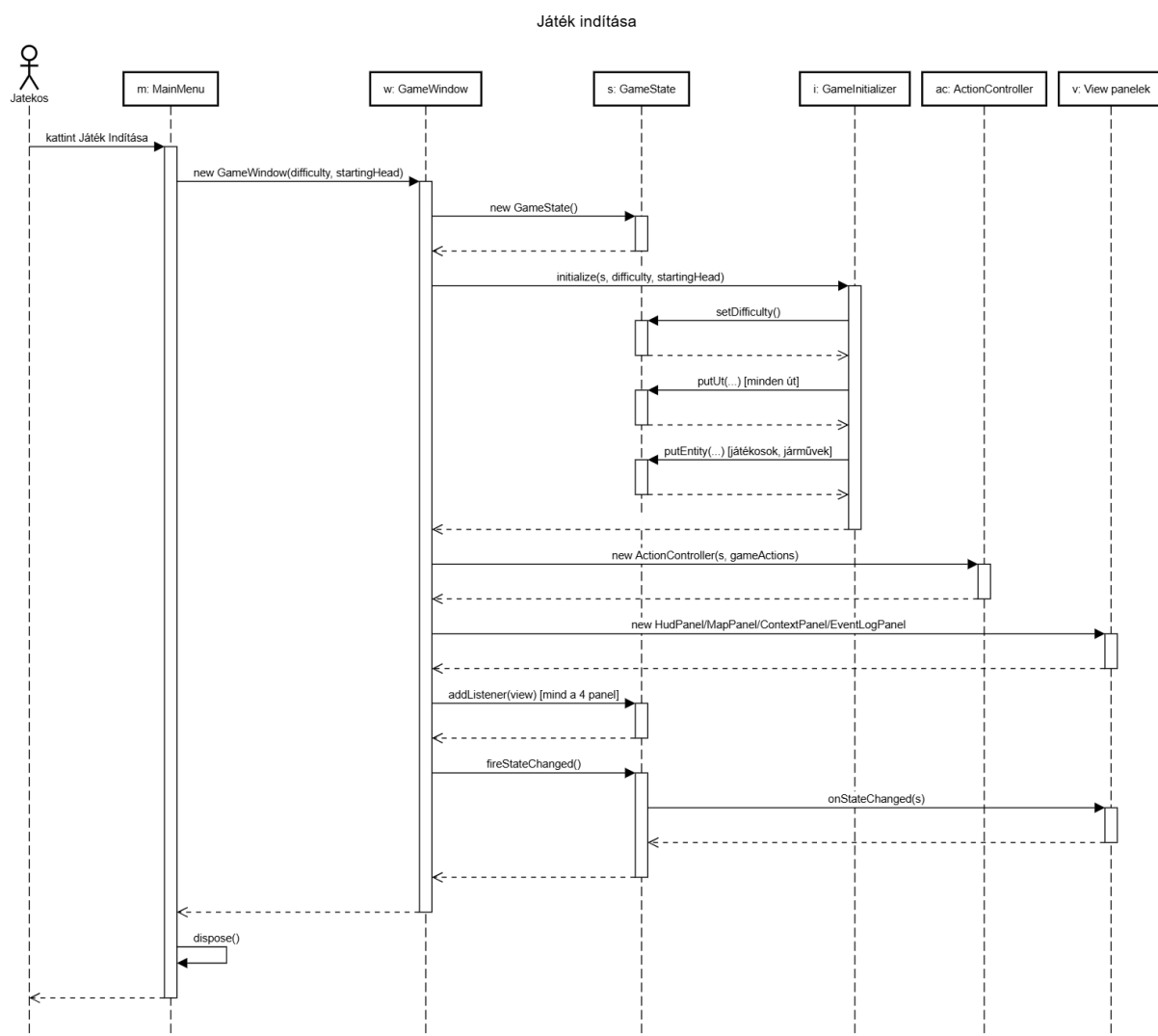
11.4. ábra. Hókotró takarítás



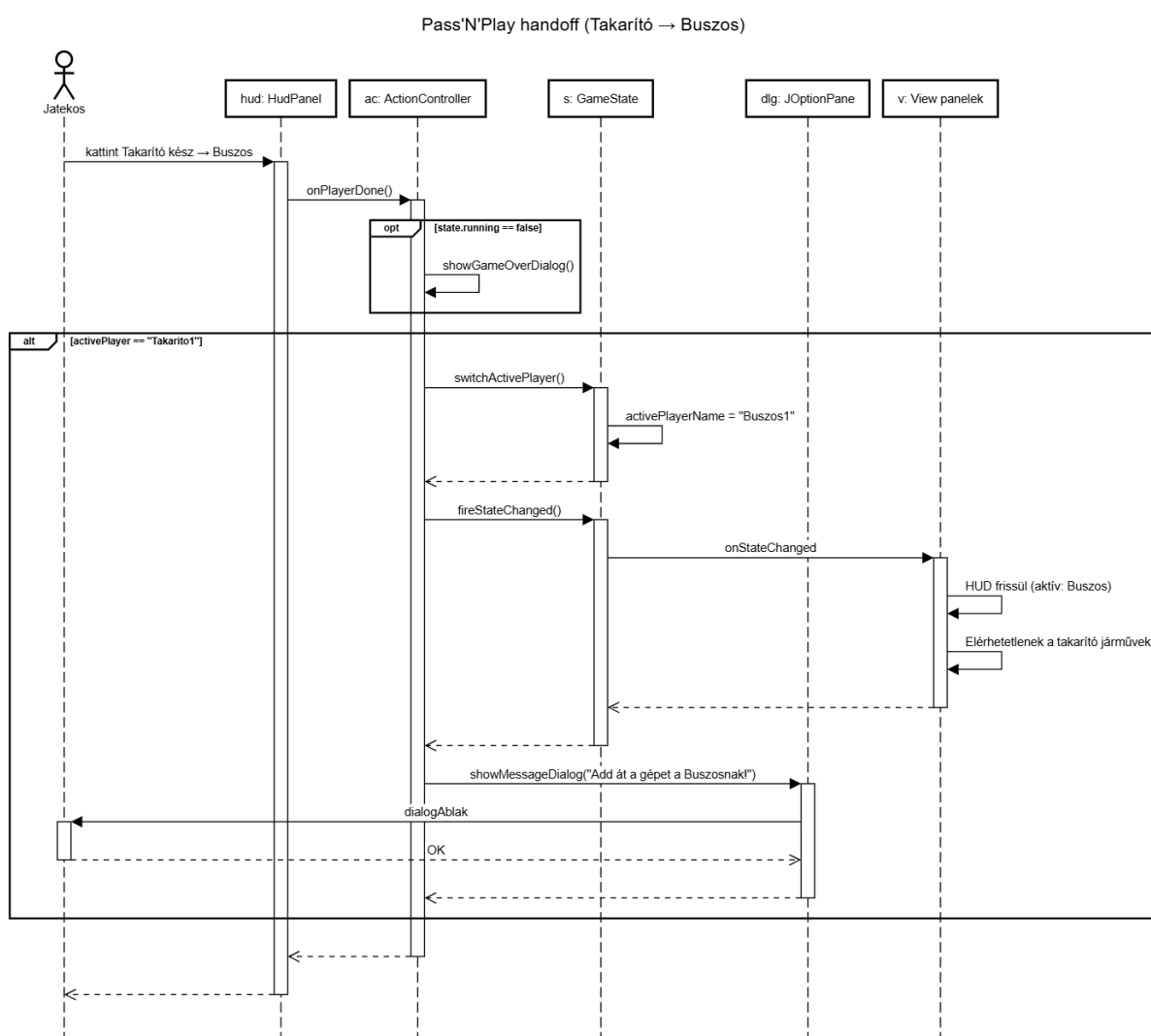
11.5. ábra. Jármű kijelölés



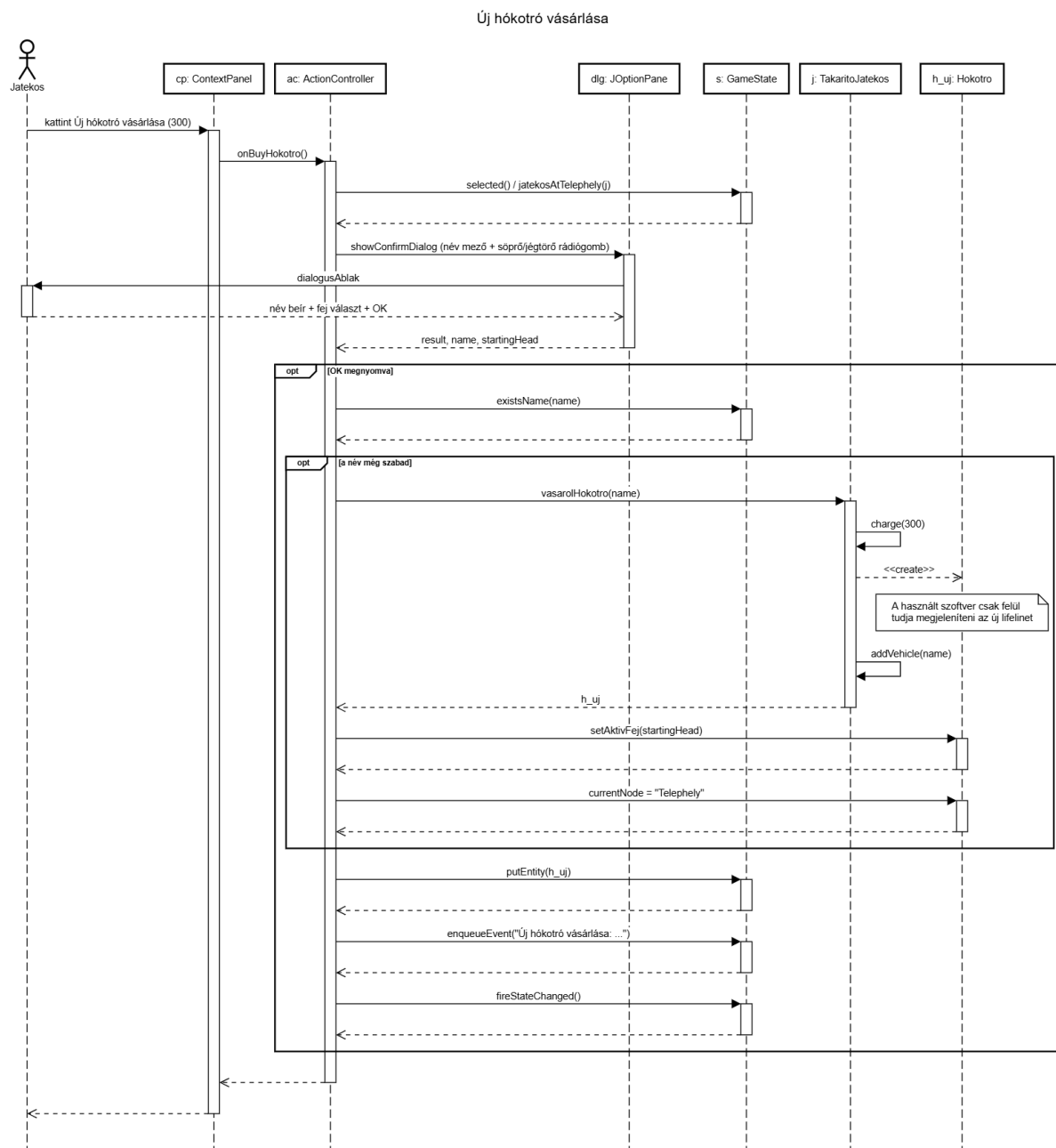
11.6. ábra. Jármű mozgás



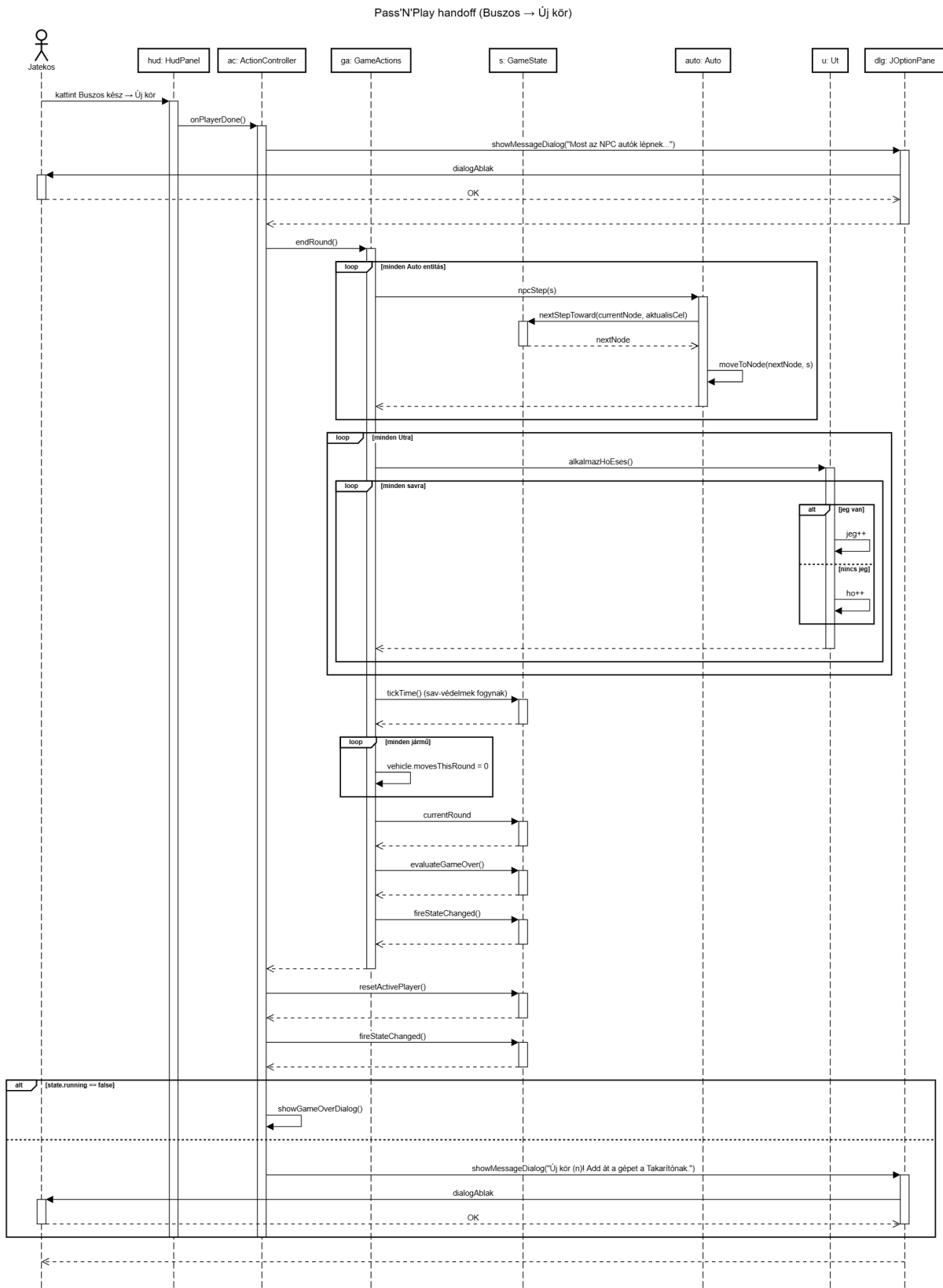
11.7. ábra. Játék indítás



11.8. ábra. Hand off



11.9. ábra. Új hókotró



11.10. ábra. Új kör

11.5. Napló

A napló tartalmazza az előző beadás óta eltelt időszak történéseit időrendben. A naplóból egyértelműen ki kell derülnie, hogy az egyes anyagrészeket ki és mennyi idő alatt készítette.

A napló bejegyzésekből áll. Minden bejegyzésnek tartalmaznia kell:

- a történes kezdetének időpontját, nap-óra pontossággal
- történes időtartamát, óra felbontással
- a szereplő(k) nevét (Kérjük a szereplők VEZETÉKNEVÉT használni)
- a tevékenység leírását.

Amennyiben a tevékenységben több szereplő vesz részt, akkor aza tevékenység csak értekezlet lehet, amelynek az eredményei DÖNTÉSEK. A döntéseket precízen meg kell szövegezni (Pl.: Az X objektum Y és Z metódusainak kódját W készíti el Q határidőre). Ha a bejegyzés egyetlen személyhez kötődik, akkor meg kell adni, hogy a tevékenység milyen dologra irányul. A dolog a feladat kapcsán elkészítendő termék, amelynek a (esetleg korábban) beadott anyagban megtalálhatónak kell lenni. A naplóbejegyzés felbontásának egysége szöveges, rajzos anyag esetében az ábra, diagram, vagy kb. fél-egy oldalnyi szöveg. Kódban az egység a metódus. (Pl.: A 3. ábrán látható szekvencia-diagram kidolgozása, vagy az X objektum Y és Z metódusainak kódolása és belövése.

Kezdet	Időtartam	Résztvevők	Leírás
feb. 18. 16h	1 óra	Csapat	Értekezlet Döntés: Segédeszközök kiválasztása (git, trello, drive)